

UNINORTE · DIRECCIÓN DE PLANEACIÓN

Manual completo SNIES Monitor · Posgrado

Cómo funciona, paso a paso, el sistema que vigila automáticamente los programas de posgrado de todas las instituciones de educación superior de Colombia: desde el robot que descarga los datos del Ministerio de Educación, hasta el correo y el panel web que se generan solos.

Repositorio: dirplaneacionun/snies_monitor_posgrado · Generado a partir de una revisión completa del código fuente — julio de 2026

Manual completo — SNIES Monitor Posgrado · Uninorte

Este documento explica, en lenguaje sencillo, cómo funciona **todo** este proyecto: desde que un robot entra a la página del gobierno a descargar un Excel, hasta que llega un correo y se actualiza una página web con gráficos. Está escrito para alguien con conocimientos **básicos** de programación — cada vez que aparece un término técnico (Selenium, XPath, cron, secret, etc.) se explica la primera vez que sale, y además hay un glosario al final.

Si solo necesitas resolver algo puntual (cambiar destinatarios, cambiar la cuenta de correo, entender por qué un programa se marcó como "modificado"), puedes ir directo a esa sección con el índice — no hace falta leer todo de corrido.

Índice

1. ¿Qué es este proyecto y para qué sirve?
 2. El flujo completo, de principio a fin
 3. Mapa de carpetas y archivos
 4. Paso 1 — Cómo se descarga la información (web scraping)
 5. Paso 2 — Qué información se guarda de cada programa
 6. Paso 3 — Cómo se decide si un programa es Nuevo, Inactivo o Modificado
 7. Paso 4 — Cómo se guarda el historial
 8. Paso 5 — El correo automático
 9. Paso 6 — El Dashboard (la página web)
 10. GitHub Actions: el robot que hace todo esto solo
 11. Los "Secrets" de GitHub
 12. Cómo cambiar los destinatarios del correo
 13. Cómo cambiar la cuenta de correo que envía los reportes
 14. GitHub Pages: cómo se publica la página web
 15. Cómo correr todo esto en tu propio computador
 16. Problemas comunes y cómo resolverlos
 17. Cosas importantes que hay que saber (detalles no obvios)
 18. Glosario de términos técnicos
-

1. ¿Qué es este proyecto y para qué sirve?

El **SNIES** (Sistema Nacional de Información de la Educación Superior) es una base de datos pública del Ministerio de Educación de Colombia donde **todas** las instituciones de educación superior del país deben registrar sus programas académicos — pregrado y posgrado. Cualquiera puede consultarla en:

<https://hecaa.mineduacion.gov.co/consultaspublicas/programas>

El problema es que esa página no avisa cuando algo cambia. Si una universidad abre una nueva maestría, le sube el valor de la matrícula a una especialización, o da de baja un doctorado, **nadie te lo notifica** — habría que entrar manualmente y comparar a ojo entre miles de programas de posgrado activos en el país.

Este proyecto (`snies_monitor_posgrado`) automatiza justamente eso, para que la Dirección de Planeación de Uninorte pueda vigilar la oferta de posgrados de la competencia sin trabajo manual:

1. Periódicamente, un robot entra a la página del SNIES y descarga el Excel con todos los programas de **posgrado activos** en el país (especializaciones, maestrías, doctorados — todos los niveles de formación de posgrado juntos).
2. Lo compara contra la descarga anterior más reciente.
3. Detecta tres tipos de novedades: programas **nuevos**, programas que quedaron **inactivos**, y programas que fueron **modificados** (cambiaron de modalidad, créditos, costo de matrícula, municipio de oferta o nivel de formación).
4. Guarda esos hallazgos acumulados en archivos Excel (para no perder el historial).
5. Envía un correo con el resumen y dos gráficos a la lista de personas interesadas.
6. Publica un panel visual (dashboard) en internet con gráficos y tablas filtrables.

Todo esto corre solo, sin que nadie tenga que prender su computador — vive en la nube usando **GitHub Actions** (se explica a fondo en la sección 10).

Este mismo proyecto tiene un "hermano" independiente que vigila **pregrado** (repositorio [snies_monitor](#) , no el que trata este manual). Comparten el mismo diseño general, pero son dos repositorios de código separados, cada uno con sus propios secretos, su propio cronograma y su propia página web publicada. Este manual solo describe el repositorio de **posgrado**.

2. El flujo completo, de principio a fin

Así se ve el proceso completo, en orden, cada vez que se ejecuta:

1. GitHub Actions "despierta" el robot (cron semanal o manual)

2. scripts/run_snies_posgrado.py

- a) Abre un Chrome invisible ("headless") y entra al portal SNIES
- b) Aplica los filtros: Institución activa + Programa activo + Nivel académico = Posgrado
- c) Descarga el Excel "Programas.xlsx"
- d) Lo archiva como Programas/Programas postgrado DD-MM-YY.xlsx
- e) Lo compara contra el archivo más reciente anterior en Programas/
- f) Clasifica: Nuevos / Inactivos / Modificados
- g) Acumula los resultados en data/novedades/*.xlsx
- h) Genera 2 gráficos (analisis_historico_posgrado.py)
- i) Envía el correo (scripts/send_report_posgrado.py)

3. GitHub Actions "commitea" (guarda) los datos nuevos en el repo, pero todavía NO los sube a GitHub (ver ⚠ sección 17, punto 3)

4. docs/generar_dashboard.py

Lee todo el historial y regenera las páginas HTML del dashboard

5. GitHub Actions guarda docs/ y recién ahí hace UN SOLO git push que sube tanto los datos nuevos como el dashboard regenerado, publicándose automáticamente en GitHub Pages

Cada una de estas cajas se explica a fondo en las secciones siguientes.

3. Mapa de carpetas y archivos

```
SNIES-POSGRADOS/
├── scripts/
│   ├── run_snies_posgrado.py → El cerebro: descarga, compara, clasifica, guarda
│   ├── send_report_posgrado.py → Arma y envía el correo
│   └── test_pipeline.py → Prueba el pipeline con Excel ya descargados,
│       sin usar Selenium (ver sección 15)
├── analisis_historico_posgrado.py → Genera los 2 gráficos que van en el correo
├── docs/
│   ├── generar_dashboard.py → Genera todo el HTML del dashboard
│   ├── index.html → Página principal (SE GENERA SOLA, no editar a mano)
│   ├── nuevos.html → Detalle de programas nuevos (SE GENERA SOLA)
│   ├── inactivos.html → Detalle de programas inactivos (SE GENERA SOLA)
│   ├── modificados.html → Detalle de programas modificados (SE GENERA SOLA)
│   ├── modificados_creditos.html → Análisis a fondo de cambios de créditos
│   └── modificados_costos.html → Análisis a fondo de cambios de matrícula
├── data/
│   ├── novedades/ → Aquí se ACUMULA todo el historial de hallazgos
│   │   ├── Nuevos_posgrado.xlsx
│   │   ├── Inactivos_posgrado.xlsx
│   │   ├── Modificados_posgrado.xlsx
│   │   ├── grafico_novedades_posgrado.png
│   │   └── grafico_modificados_unicos_por_division_posgrado.png
├── Programas/ → Un Excel "foto" (snapshot) por cada corrida,
│   con el nombre "Programas postgrado DD-MM-YY.xlsx"
│   (fíjate: dice "postgrado", con "t" – ver sección 17).
│   Este es el historial crudo, sin procesar.
├── "Categorización divisiones SNIES .xlsx" → Tabla que traduce "área de conocimiento
│   SNIES" → "a qué facultad/división de Uninorte compete". Vive en la RAÍZ del
│   repositorio (no dentro de data/), y su nombre real tiene un espacio antes
│   de ".xlsx" – ver sección 17.
├── .github/workflows/
│   └── snies_posgrado_daily.yml → La configuración de "cuándo y cómo correr esto solo"
├── pregrado/ → Carpeta de SOLO CONSULTA con HTML viejos del
│   dashboard de pregrado. Está en .gitignore, no es
│   parte de este proyecto ni se usa para nada.
├── requirements.txt → Lista de librerías de Python necesarias
└── (no hay README.md en este repositorio – este manual cumple ese rol)
```

Regla de oro: todo archivo dentro de `docs/*.html` se **reescribe automáticamente** cada vez que corre `generar_dashboard.py`. Si alguien edita `docs/index.html` a mano, ese cambio se pierde en la próxima corrida. Para cambiar cómo se ve la página hay que editar `docs/generar_dashboard.py`, no el HTML final.

4. Paso 1 — Cómo se descarga la información (web scraping)

4.1 ¿Qué es "hacer scraping"?

"Scraping" es la técnica de programar un robot que **usa un navegador de internet como lo usaría una persona**: entra a una página, hace clic en botones, llena filtros, espera a que cargue, y descarga archivos. Se usa cuando la página no ofrece una forma más directa (como una API) de obtener los datos.

Este proyecto usa una librería de Python llamada **Selenium**, que controla una copia real de Google Chrome. En el servidor de GitHub corre en modo **"headless"** (sin ventana visible) — literalmente no hay pantalla, pero el navegador funciona igual por dentro.

```
# scripts/run_snies_posgrado.py
opts.add_argument("--headless=new")          # Chrome invisible
opts.add_argument("--window-size=1920,1080") # aunque sea invisible, simula una pantalla grande
```

La librería `webdriver-manager` se encarga de descargar automáticamente la versión correcta de "ChromeDriver" (el conector entre Selenium y Chrome) — nadie tiene que instalarlo a mano.

4.2 La página y los filtros que se aplican

La página que se visita es:

```
SNIES_URL = "https://hecaa.mineduacion.gov.co/consultaspublicas/programas"
```

Es la **misma** página que usa el monitor de pregrado — la diferencia está en qué filtros se seleccionan. El portal permite filtrar los programas usando **botones de radio** (los circulitos donde solo puedes elegir una opción). El robot de posgrado selecciona, en este orden, estos tres filtros:

FILTRO QUE HACE CLIC	QUÉ SIGNIFICA	POR QUÉ SE USA
"Activo" (institución, coincidencia exacta)	La institución educativa está activa (no cerrada/liquidada)	No tiene sentido vigilar universidades que ya no existen
"Activo (" (programa)	El programa académico específico está activo (no cancelado)	Solo interesan programas que se pueden ofertar hoy
"Posgrado ("	Nivel académico = posgrado (no pregrado)	Este pipeline solo vigila posgrado

En código (simplificado):

```
_pf_select_radio(driver, "Activo", exact=True) # institución activa
_wait_ajax(driver)
_pf_select_radio(driver, "Activo ("           # programa activo
_wait_ajax(driver)
_pf_select_radio(driver, "Posgrado ("         # nivel académico = posgrado
_wait_ajax(driver)
```

A diferencia del monitor de pregrado (que además filtra por "Universitario" para excluir técnico/tecnológico), acá **no se toca el filtro de "Nivel de Formación"** — se deja en su valor por defecto, "Todos". Esto es intencional: dentro de posgrado el SNIES distingue varios "niveles de formación" (Especialización, Especialización Médico-Quirúrgica, Maestría, Doctorado...) y el panel del portal tiene *varios* controles distintos que dicen literalmente "Todos" para categorías distintas (Estado de la institución, Tipo de sede, etc.). El comentario en el código lo explica así:

```
# "Nivel Formación: Todos" es el default - no se toca para evitar ambigüedad
# con los otros labels "Todos" del panel (Estado Institución, Tipo de sede, etc.)
```

En otras palabras: como buscar un botón que diga "Todos" podría encontrar el botón equivocado (hay varios en la misma pantalla), es más seguro y más simple **no tocarlo** y confiar en que ese filtro ya viene en "Todos" por defecto. Esto significa que el Excel descargado trae **todos los niveles de posgrado mezclados** (especializaciones, maestrías, doctorados, etc.) en un solo archivo — la columna `NIVEL_DE_FORMACIÓN` es la que permite distinguir cada uno dentro de los datos ya descargados.

¿Por qué se buscan los botones por su texto ("Activo", "Posgrado") y no por un identificador técnico?

Las páginas de sistemas del gobierno colombiano suelen estar hechas con una tecnología llamada JSF/PrimeFaces, que le pone a cada botón un identificador (`id`) interno que cambia cada vez que el Ministerio actualiza el portal. Si el robot buscara por ese `id` , dejaría de funcionar con cualquier actualización menor de la página. En cambio, el texto visible ("Activo", "Posgrado") es mucho menos probable que cambie, así que el robot es más resistente a rediseños del portal. La función que hace esto se llama `_pf_select_radio` y funciona ubicando la etiqueta (`<label>`) con ese texto, y desde ahí activando el botón de radio invisible que tiene asociado.

Una vez aplicados los 3 filtros, el robot hace clic en el botón de descarga, que también se busca por su texto visible:

```
XPATHS = {  
    "descarga": '//button[.//span[normalize-space()="Descargar programas"]]',  
}
```

Esto es un **XPath**: una forma estándar de decirle al navegador "búscame el botón que contiene un texto que diga exactamente 'Descargar programas'".

4.3 Esperar la descarga

Después de pedir la descarga, el robot no sabe de antemano cuánto se va a demorar el servidor del gobierno en generar el archivo. Por eso hace lo siguiente:

```
DOWNLOAD_TIMEOUT = 120 # segundos máximos esperando la descarga  
  
elapsed = 0  
while elapsed < DOWNLOAD_TIMEOUT:  
    time.sleep(5)  
    elapsed += 5  
    if expected_file.exists() and not partial_file.exists():  
        break # ¡ya llegó completo!  
else:  
    raise TimeoutError("Archivo no apareció tras 120s...")
```

Cada 5 segundos revisa si ya apareció el archivo `Programas.xlsx` completo (mientras se está descargando, Chrome lo llama temporalmente `Programas.crdownload`). Si pasan 2 minutos sin que aparezca, el robot se rinde y reporta el error — normalmente esto pasa si el portal del gobierno está caído, muy lento, o cambió de diseño.

4.4 Capturas de pantalla para depurar

Como el navegador corre invisible en el servidor de GitHub, es imposible "verlo" en vivo si algo sale mal. Por eso el robot se toma dos fotos y las guarda en `tmp/` :

```
driver.save_screenshot(str(TMP_DIR / "debug_snies.png")) # justo al abrir la página  
driver.save_screenshot(str(TMP_DIR / "debug_post_filtros.png")) # después de aplicar los filtros
```

Estas capturas **no** se guardan en el repositorio (la carpeta `tmp/` está en `.gitignore`), pero el workflow de GitHub Actions tiene un paso dedicado que las sube como "artifact" (archivo descargable adjunto a la corrida) cada vez, incluso si el resto del proceso falla:

```
- name: Subir screenshots de debug  
  if: always()  
  uses: actions/upload-artifact@v4  
  with:  
    name: debug-screenshots  
    path: tmp/*.png  
    retention-days: 3
```

`if: always()` quiere decir "haz esto pase lo que pase en los pasos anteriores, incluso si fallaron". Estos artifacts se pueden descargar desde la pestaña **Actions** de GitHub, abriendo la corrida en cuestión, y se borran solos a los 3 días.

4.5 El "candado de seguridad" contra descargas corruptas

Antes de aceptar el Excel descargado (o el snapshot anterior) como válido para comparar, el pipeline hace una validación de sentido común: un archivo de **posgrado activo** en Colombia normalmente tiene entre 8.000 y 11.000 programas (según el comentario del propio código). Si cualquiera de los dos archivos (el de hoy o el anterior) tiene más de 12.000 filas, es señal de que **algo salió mal con los filtros** — por ejemplo, se descargó sin filtrar, o mezclado con pregrado:

```
UMBRAL = 12_000
if len(df_hoy) > UMBRAL:
    log.error(f"Snapshot HOY tiene {len(df_hoy)} programas — demasiados para ser "
            "solo posgrado activo. Probable descarga sin filtros. Abortando comparación.")
    raw_file.unlink(missing_ok=True)
    return vacio # no compara nada, no daña el historial acumulado
```

Si esto pasa, el pipeline **no** compara ni acumula nada ese día, y **borra** el archivo recién descargado (para no dejarlo archivado como si fuera válido) — mejor no reportar novedades que reportar cientos de falsos positivos por una descarga corrupta.

5. Paso 2 — Qué información se guarda de cada programa

Del Excel descargado, el sistema no guarda todas las columnas — solo estas, que son las relevantes para el monitoreo (lista `BASE_COLS` en el código):

COLUMNA	QUÉ ES, EN PALABRAS SIMPLES
CÓDIGO_INSTITUCIÓN	Número único que identifica a la institución educativa
NOMBRE_INSTITUCIÓN	Nombre de la institución
SECTOR	Oficial (pública) o Privado
DEPARTAMENTO_OFERTA_PROGRAMA	En qué departamento de Colombia se dicta el programa
MUNICIPIO_OFERTA_PROGRAMA	En qué ciudad/municipio se dicta el programa
CÓDIGO_SNIES_DEL_PROGRAMA	El identificador único del programa. Es como la "cédula" de cada programa — nunca cambia mientras el programa exista. Todo el sistema de comparación se basa en este número.
NOMBRE_DEL_PROGRAMA	Ej: "MAESTRÍA EN ADMINISTRACIÓN"
MODALIDAD	Presencial, Virtual, A distancia, Dual, o combinaciones ("Híbrida...")
NÚMERO_CRÉDITOS	Cuántos créditos académicos tiene el programa
NÚMERO_PERIODOS_DE_DURACIÓN	Cuántos semestres/periodos dura el programa
PERIODICIDAD	Semestral, anual, etc.
COSTO_MATRÍCULA_ESTUD_NUEVOS	Valor de la matrícula para estudiantes que ingresan por primera vez
PERIODICIDAD_ADMISIONES	Cada cuánto abre admisiones el programa
FECHA_DE_REGISTRO_EN_SNIES	Cuándo se registró el programa en el sistema del Ministerio
CINE_F_2013_AC_CAMPO_AMPLIO / ..._ESPECÍFIC / ..._DETALLADO	Clasificación internacional de la UNESCO por área del conocimiento (Educación, Salud, Ingeniería, etc.), en 3 niveles de detalle, de lo más general a lo más específico
NÚCLEO_BÁSICO_DEL_CONOCIMIENTO	Otra clasificación de áreas de conocimiento, propia de Colombia
NIVEL_DE_FORMACIÓN	El nivel de posgrado exacto: Especialización, Especialización Médico-Quirúrgica, Maestría, Doctorado, etc.

Con la columna `CINE_F_2013_AC_CAMPO_DETALLADO` se hace un cruce contra el archivo `Categorización divisiones SNIES .xlsx` (hoja `Hoja3`, ubicado en la raíz del repositorio), que traduce cada área de conocimiento a **"¿qué facultad/división de Uninorte compete con este programa?"** (columna resultante `DIVISIÓN UNINORTE`):

```
def load_categorizacion() -> pd.DataFrame:
    return (
        pd.read_excel(CAT_FILE, sheet_name="Hoja3")[
            ["CINE_F_2013_AC_CAMPO_DETALLADO", "DIVISIÓN UNINORTE"]
        ]
        .drop_duplicates()
    )
```

Si un programa tiene un área CINE que **no** está en esa tabla de traducción, el sistema no lo descarta — simplemente lo etiqueta como `"Sin clasificar"` en vez de asignarle una división:

```
df["DIVISIÓN UNINORTE"] = df["DIVISIÓN UNINORTE"].fillna("Sin clasificar")
```

Así, si en el correo o el dashboard ves programas marcados como "Sin clasificar", significa que su área CINE todavía no está mapeada en ese archivo Excel de categorización — hay que abrirlo y agregar la fila correspondiente si se quiere que ese tipo de programa aparezca bajo una división real.

Además, al cargar cada Excel se le hace una limpieza de datos: se quitan las dos últimas filas (que son un aviso legal del SNIES, no un programa real), se convierte el código SNIES a número entero (descartando filas donde no se pudo convertir), los créditos a número entero, y la fecha de

registro a una fecha real.

6. Paso 3 — Cómo se decide si un programa es Nuevo, Inactivo o Modificado

6.1 La idea de "snapshot" (foto) y el código SNIES como huella digital

Cada vez que el robot descarga el Excel, esa descarga es una **"foto" (snapshot)** del estado del SNIES en ese momento exacto. El sistema guarda esa foto permanentemente en `Programas/Programas postgrado DD-MM-YY.xlsx` y luego la compara contra **la foto anterior más reciente** que exista en esa carpeta (buscando, entre todos los archivos con ese patrón de nombre, la fecha más reciente que sea anterior a hoy).

Como cada programa tiene un `CÓDIGO_SNIES_DEL_PROGRAMA` único que nunca cambia, comparar dos fotos es tan simple como comparar dos listas (conjuntos) de códigos:

```
snies_hoy = set(df_hoy["CÓDIGO_SNIES_DEL_PROGRAMA"]) # códigos que aparecen HOY
snies_ant = set(df_ant["CÓDIGO_SNIES_DEL_PROGRAMA"]) # códigos que aparecían ANTES
```

6.2 Programa Nuevo

Un programa es "Nuevo" si su código aparece en la foto de hoy, pero **no** aparecía en la foto anterior:

```
nuevosDF = df_hoy[df_hoy["CÓDIGO_SNIES_DEL_PROGRAMA"].isin(snies_hoy - snies_ant)]
```

En español: "todos los programas de hoy cuyo código no estaba en la lista de antes".

6.3 Programa Inactivo

Es el caso contrario: el código estaba en la foto anterior, pero **ya no aparece** en la de hoy:

```
inactivosDF = df_ant[df_ant["CÓDIGO_SNIES_DEL_PROGRAMA"].isin(snies_ant - snies_hoy)]
```

Ojo: como el filtro de descarga solo trae programas **activos**, un programa puede "desaparecer" de la lista tanto porque lo cerraron de verdad, como porque el Ministerio lo pasó a estado inactivo/suspendido en el SNIES. Para el reporte, ambos casos se ven igual: dejó de estar activo. De hecho, cada fila de nuevos/inactivos/modificados queda además marcada con una columna `Estado` (`"Activo"` o `"Inactivo"`), según si su código sigue existiendo hoy) — aunque, vale la pena saberlo, esa columna **no se muestra** ni en el correo ni en el dashboard; solo queda guardada dentro de los archivos Excel de `data/novedades/` por si alguien los abre directamente.

6.4 Programa Modificado — las 5 variables que se vigilan

Este es el caso más interesante. Un programa se marca como "Modificado" cuando su código existe **en ambas fotos** (hoy y antes — o sea, sigue activo) **pero al menos uno** de estos 5 campos cambió de valor:

```
COLS_VIGILAR = [
    "MODALIDAD",
    "NÚMERO_CRÉDITOS",
    "COSTO_MATRÍCULA_ESTUD_NUEVOS",
    "MUNICIPIO_OFERTA_PROGRAMA",
    "NIVEL_DE_FORMACIÓN",
]
```

CAMPO VIGILADO	QUÉ SIGNIFICA QUE CAMBIE
MODALIDAD	El programa pasó de Presencial a Virtual, o agregó una modalidad híbrida, etc.
NÚMERO_CRÉDITOS	Le subieron o bajaron los créditos académicos
COSTO_MATRÍCULA_ESTUD_NUEVOS	Cambió el valor de la matrícula para estudiantes nuevos
MUNICIPIO_OFERTA_PROGRAMA	El programa se empezó a ofrecer en otra ciudad, o cambió de sede
NIVEL_DE_FORMACIÓN	El programa cambió de nivel de posgrado — por ejemplo, de "Especialización" pasó a estar registrado como "Maestría"

Para cada programa común a ambas fotos, el sistema arma una tabla donde pone lado a lado el valor de hoy y el valor de antes de **todas** las columnas de **BASE_COLS** (usando sufijos **_NUEVO** y **_ANTIGUO**, que después del análisis se renombran a sin-sufijo y **_ANTERIOR**), y marca como "modificado" cualquier fila donde alguno de esos **5** pares vigilados sea diferente:

```
comparativa = df_com_hoy.merge(df_com_ant, on="CÓDIGO_SNIES_DEL_PROGRAMA",
                              suffixes=("_NUEVO", "_ANTIGUO"))

mascara = pd.Series(False, index=comparativa.index)
for col in COLS_VIGILAR:
    col_n, col_a = f"{col}_NUEVO", f"{col}_ANTIGUO"
    mascara |= (comparativa[col_n].fillna("").astype(str)
               != comparativa[col_a].fillna("").astype(str))

modificadosDF = comparativa[mascara]
```

Además, el sistema genera automáticamente una columna **QUE_CAMBIO** que explica en texto legible **exactamente qué cambió y de qué valor a qué valor**:

```
def _que_cambio(row) -> str:
    partes = []
    for col in COLS_VIGILAR:
        val_n, val_a = str(row[f"{col}_NUEVO"]).strip(), str(row[f"{col}_ANTIGUO"]).strip()
        if val_n != val_a:
            partes.append(f"{col}: {val_a} → {val_n}")
    return " | ".join(partes) if partes else "Cambio en otros campos"
```

Ejemplo (ilustrativo, con el mismo formato que produce el sistema real):

Una **Maestría en Finanzas** de una universidad privada aparece un mes con:

QUE_CAMBIO = "NÚMERO_CRÉDITOS: 48 → 44 | COSTO_MATRÍCULA_ESTUD_NUEVOS: 9500000 → 10200000"

Es decir: le bajaron 4 créditos al programa **y** al mismo tiempo le subieron la matrícula. Si además hubiera cambiado de modalidad, el texto simplemente agregaría un tercer segmento separado por " | " .

6.5 ⚠ Importante: qué cambios el sistema NO detecta como "Modificado"

Esto es clave para entender los límites del monitor. **Solo se vigilan esos 5 campos**. Si un programa cambia de **nombre**, de **institución dueña**, de **duración en periodos** (**NÚMERO_PERIODOS_DE_DURACIÓN**), de **núcleo básico del conocimiento**, de **departamento** (a diferencia de municipio, el departamento no está vigilado), o de cualquier otra columna que no esté en **COLS_VIGILAR**, el sistema **no lo va a reportar como "Modificado"** — para el comparador, ese programa sigue "igual", aunque en la realidad algo haya cambiado.

Esto es una decisión de diseño (probablemente para no generar demasiado ruido con cambios menores), pero es importante que quien lea los reportes sepa que existe este punto ciego. Si en algún momento se quiere vigilar un campo adicional, basta con agregarlo a la lista **COLS_VIGILAR** en **scripts/run_snies_posgrado.py** (alrededor de la línea 90) — no hay que tocar nada más: el resto del sistema (correo, dashboard) ya sabe leer cualquier campo nuevo que aparezca en **QUE_CAMBIO** .

Un detalle fino que vale la pena conocer: aunque `NÚMERO_PERIODOS_DE_DURACIÓN` no está en `COLS_VIGILAR` y por lo tanto **no dispara** por sí solo la marca de "Modificado", su valor de antes y de después sí queda **guardado** en el Excel de modificados (porque el cruce se hace con todas las columnas de `BASE_COLS`, no solo las vigiladas). El dashboard aprovecha justamente ese dato "de regalo" en sus páginas de análisis de créditos y costos, para mostrar si la duración también cambió cuando otro campo fue el que disparó la marca de "modificado".

6.6 Una protección extra: evitar comparaciones "fantasma"

A veces el portal del SNIES devuelve accidentalmente el mismo código de programa duplicado dentro de un mismo archivo (por errores del portal, o por variantes sin código distinto). Si eso pasara y no se controlara, al cruzar la foto de hoy con la de antes se generarían combinaciones falsas (por ejemplo, 2 filas de hoy × 3 filas de antes = 6 "modificaciones" en vez de 1). El código detecta esos duplicados, avisa por log (`"Snapshot HOY tiene N código(s) duplicado(s)"`), y se queda solo con la primera aparición de cada código antes de comparar — así nunca se inventan modificaciones que no existen.

7. Paso 4 — Cómo se guarda el historial

7.1 Los snapshots crudos, en `Programas/`

Cada corrida (si logró descargar el archivo con éxito) archiva una copia completa y sin procesar del Excel descargado ese día:

```
archive_path = PROGRAMAS_DIR / f"Programas postgrado {today.strftime('%d-%m-%y')}.xlsx"
```

Ojo con la ortografía: el nombre del archivo dice **"postgrado"** (con "t"), no "posgrado" — esto es así en todo el código y hay una expresión regular (`_PROG_RE`) que depende exactamente de ese patrón para poder identificar y ordenar los snapshots por fecha. Si alguna vez alguien archiva un Excel a mano en esa carpeta, el nombre debe respetar ese patrón exacto (incluyendo el año en dos dígitos) para que el sistema lo reconozca como snapshot válido.

Esta carpeta es el **corazón real del historial**: de ahí sale tanto la comparación de cada corrida ("¿cuál es la foto anterior más reciente?") como todos los gráficos de evolución histórica del dashboard (el total de programas activos a través del tiempo, por ejemplo). **No conviene borrar ni mover archivos de `Programas/` manualmente.**

7.2 El acumulado de novedades, en `data/novedades/`

Cada corrida **no reemplaza** los resultados anteriores — los **acumula**. Los tres archivos (`Nuevos_posgrado.xlsx` , `Inactivos_posgrado.xlsx` , `Modificados_posgrado.xlsx`) van creciendo corrida tras corrida:

```
def acumular(existing_path: Path, nuevo_df: pd.DataFrame) -> pd.DataFrame:
    dedup_cols = ["CÓDIGO_SNIES_DEL_PROGRAMA", "FECHA_OBTENCION"]
    if existing_path.exists():
        existing = pd.read_excel(existing_path)
        if nuevo_df.empty:
            return existing
        combined = pd.concat([existing, nuevo_df], ignore_index=True)
        return combined.drop_duplicates(subset=dedup_cols, keep="last")
    return nuevo_df
```

La deduplicación es por `CÓDIGO_SNIES_DEL_PROGRAMA` + `FECHA_OBTENCION` (la fecha en que se detectó la novedad): esto evita que si el robot corre dos veces el mismo día, se dupliquen las filas — pero si el mismo programa vuelve a modificarse en una corrida de otro día, sí queda registrado de nuevo (con otra fecha), porque es información nueva.

Importante — estas tablas **NO** son "solo la corrida más reciente". Son el acumulado de *toda* la historia del monitor desde que arrancó. Cuando se quiera saber "¿qué pasó en la última corrida?", hay que fijarse en la columna `FECHA_OBTENCION` (o usar el filtro por fecha del dashboard, o las tarjetas "último run" del índice), no asumir que el archivo entero corresponde a lo último que pasó.

Nota práctica: si algún día hay que reconstruir el historial de "novedades" desde cero, basta con borrar (o vaciar) los archivos de `data/novedades/` — el pipeline los vuelve a crear automáticamente la próxima vez que corra, comparando desde donde tenga snapshots en `Programas/` en adelante.

8. Paso 5 — El correo automático

Al final de cada corrida, `run_snies_posgrado.py` genera los gráficos (`analisis_historico_posgrado.py`) y llama a `send_report_posgrado.py` , que construye un correo en formato HTML y lo envía por **SMTP** (el protocolo estándar de internet para enviar correos por programación).

8.1 Qué contiene el correo

- Un título con la fecha del reporte (`Reporte Mensual SNIES – Posgrado – DD/MM/AAAA` — sobre por qué dice "Mensual", ver sección 17).
- Una sección **Programas Nuevos**, con una tabla de hasta **10 filas** (nombre del programa, institución, nivel de formación, departamento de oferta, división Uninorte). Si hubo más de 10 nuevos ese día, el correo lo indica con un texto tipo "... y *N* registro(s) más en el adjunto Excel".
- Una sección **Programas Inactivos**, con el mismo formato (máximo 10 filas visibles en el cuerpo del correo).
- Una sección **Programas Modificados**, igual, incluyendo la columna `QUE_CAMBIO` .
- Importante:** a diferencia de lo que uno podría esperar, **las tres tablas del correo siempre se recortan a 10 filas** — incluso la de "Nuevos". Si un día hay, por ejemplo, 40 programas nuevos, el correo solo muestra 10 en el cuerpo del mensaje; los 40 completos solo se ven abriendo el Excel adjunto.
- Dos **gráficos en PNG** adjuntos (no van incrustados dentro del cuerpo del correo, sino como archivos adjuntos aparte):
 - Nuevos vs. Inactivos por División Uninorte (acumulado histórico).
 - Top divisiones Uninorte con más programas *únicos* modificados.
- Los **tres archivos Excel completos** de `data/novedades/` adjuntos, con todo el historial acumulado (no solo lo de esa corrida) — así quien reciba el correo puede filtrar/analizar todo a fondo en Excel si quiere.

```
# scripts/send_report_posgrado.py
adjuntos = sorted(NOVEDADES_DIR.glob("*.xlsx"))
for path in adjuntos:
    # ... adjunta cada Excel encontrado en data/novedades/ al correo
```

Nota: ese `glob("*.xlsx")` adjunta **cualquier** Excel que exista en `data/novedades/` en ese momento, no solo los tres esperados — si alguna vez se guarda ahí un archivo de prueba o temporal, también saldría adjunto en el próximo correo real.

8.2 De dónde saca las credenciales

El correo **nunca tiene contraseñas escritas en el código**. Las lee de variables de entorno (información que se le pasa al programa desde afuera, no desde el archivo mismo), que en producción vienen de los *Secrets* de GitHub (sección 11):

```
smtp_user    = os.environ["SMTP_USER"]           # ej: monitor.snies@gmail.com
smtp_pass    = os.environ["SMTP_PASS"]           # contraseña de aplicación
destinatarios = [d.strip() for d in os.environ["DESTINATARIOS"].split(",")]
```

8.3 Por dónde sale el correo

```
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587
```

Actualmente el correo sale por **Gmail**. Se conecta, se autentica con usuario/contraseña (o contraseña de aplicación), y envía.

8.4 Si algo falla, no se cae todo el proceso

Tanto la generación de gráficos como el envío de correo están protegidos con manejo de errores (`try/except` con `log.exception`) en `main()` : si por lo que sea el correo no se pudo enviar (contraseña vencida, Gmail bloqueó el acceso, etc.) o los gráficos no se pudieron generar, el error se registra en el log del workflow **pero el resto del pipeline sigue** — los datos ya se guardaron en `data/novedades/` y `Programas/` de todas formas, y el dashboard se genera igual. Es decir: nunca se pierde información de programas por un fallo de correo; en el peor caso solo no llega el aviso por email esa vez.

9. Paso 6 — El Dashboard (la página web)

Todo lo que se ve en la página publicada se genera automáticamente con `docs/generar_dashboard.py` , que lee **todo** el historial (`Programas/` + `data/novedades/`) y escribe archivos HTML nuevos cada vez. No hay una base de datos ni un servidor corriendo detrás — son páginas HTML "estáticas" con los datos incrustados **directamente dentro del propio archivo HTML** (como texto JSON dentro de una etiqueta `<script>`), y todo el filtrado y los gráficos pasan en el navegador de quien lo visita, con JavaScript. Por eso la página carga rápido y no necesita un "backend" corriendo en algún servidor. (A diferencia de otros proyectos parecidos, aquí **no existe** un archivo `dashboard_data.json` separado — los datos viven empacados dentro de cada página HTML.)

9.1 Página principal (`index.html`)

Lo primero que se ve son 4 tarjetas (KPIs = indicadores clave):

- **Programas activos:** el total de programas de posgrado activos hoy en Colombia (según la última foto descargada).
- **Nuevos (último run) / Inactivos (último run) / Modificados (último run):** cuántos se detectaron en la corrida más reciente, con el acumulado histórico debajo. Cada tarjeta es un botón que lleva a su página de detalle.

Debajo hay una **barra de filtros globales**, fija en la parte de arriba de la pantalla, que afecta *todas* los gráficos y tablas de la página a la vez: buscador de texto libre, sector, departamento y modalidad. El buscador ignora tildes/mayúsculas y acepta varias palabras (todas deben aparecer, en cualquier orden, en cualquier columna de la fila). El filtro de "Modalidad" agrupa automáticamente las 6 modalidades más frecuentes por nombre propio, y agrupa el resto bajo "Otras" (para que el mismo filtro tenga sentido tanto en las tablas como en el gráfico de evolución histórica).

Después vienen los gráficos (hechos con **Plotly**, una librería que dibuja gráficos interactivos: se puede pasar el mouse para ver el valor exacto, hacer zoom, etc.):

GRÁFICO	QUÉ MUESTRA
Total acumulado de programas activos	Línea con la evolución del número total de programas de posgrado activos, corrida tras corrida
Aperturas vs. cierres netos	Barras verdes (nuevos) contra barras rojas (inactivos) por periodo semestral, con una línea del neto. Si un periodo tuvo muy pocas corridas del monitor, se marca con una advertencia porque esos números no son comparables 1:1
Evolución de la modalidad	Una tarjeta con mini-gráfico por cada modalidad, ordenadas de mayor a menor crecimiento porcentual desde el primer registro
Por sector	Torta Oficial vs. Privado
Top 15 departamentos de oferta	Barras horizontales
Distribución por duración (periodos)	Barras apiladas por periodicidad — si se hace clic en una barra, se abre abajo una tabla con el listado exacto de programas activos de esa duración

Más abajo están las **pestañas** de Nuevos / Inactivos / Modificados, cada una con su tabla ordenable (clic en el encabezado de cualquier columna).

9.2 Páginas de detalle (`nuevos.html` , `inactivos.html` , `modificados.html`)

Al hacer clic en una tarjeta KPI o en una pestaña se llega a una página dedicada con **todo** el historial acumulado de ese tipo de novedad (no solo la última corrida), con su propia barra de filtros (texto, sector, departamento, institución con autocompletar, división Uninorte, modalidad/campo CINE o tipo de cambio detectado, y fecha de la corrida).

9.3 Páginas de análisis profundo: créditos y costos

`modificados_creditos.html` y `modificados_costos.html` son páginas dedicadas exclusivamente a analizar, respectivamente, los cambios de número de créditos y los cambios de costo de matrícula, con rankings, histogramas de cuánto suben o bajan (para costos se usa el cambio porcentual, no el valor absoluto en pesos, porque es más comparable entre programas de valores muy distintos), y análisis de "co-cambios" (si un programa cambió de créditos, ¿también cambió de costo o de modalidad al mismo tiempo?).

9.4 Cómo y cuándo se regenera

El dashboard se regenera **automáticamente** en cada corrida del workflow, justo después de guardar los datos nuevos. Nunca hay que generarlo a mano en producción — solo tendría sentido correrlo manualmente si se quieren **probar cambios de diseño** antes de subirlos (sección 15).

10. GitHub Actions: el robot que hace todo esto solo

10.1 ¿Qué es un "workflow", un "cron" y un "secret"?

- **GitHub Actions** es un servicio (gratuito dentro de ciertos límites) de GitHub que permite programar tareas automáticas que corren en un computador temporal en la nube ("el runner"), disparadas por un horario o manualmente.
- Un **workflow** es un archivo `.yaml` que describe, paso a paso, qué se debe hacer. En este proyecto es `.github/workflows/snies_posgrado_daily.yaml`.
- Un **cron** es la forma estándar (de Unix, con más de 40 años de antigüedad) de escribir "a qué hora y qué días correr algo", con 5 valores separados por espacio: `minuto hora día-del-mes mes día-de-la-semana`.
- Un **secret** es un valor sensible (contraseña, token) que GitHub guarda cifrado y que el workflow puede usar sin que quede visible en el código ni en los logs (sección 11).

10.2 El cronograma actual y cómo cambiarlo

```
on:
  schedule:
    - cron: '7 13 * * 2' # 08:07 hora Colombia (UTC-5), martes
  workflow_dispatch:
```

Esto se lee: minuto `7`, hora `13` (en UTC — hora universal), cualquier día del mes (`*`), cualquier mes (`*`), día de la semana `2` (martes, con domingo=0, lunes=1, martes=2...). Colombia está en UTC-5 todo el año (no cambia de horario), así que 13:07 UTC = **8:07 a.m. hora Colombia**, todos los **martes**.

El minuto `7` (en vez de un redondo `0`) es a propósito: miles de workflows en todo GitHub están programados para dispararse justo "en punto" (a las xx:00), y en esos minutos el servicio se congestiona más y a veces retrasa o incluso salta corridas. Empezar en un minuto "raro" reduce ese riesgo.

`workflow_dispatch` es lo que permite correrlo **manualmente** desde la pestaña Actions de GitHub, en cualquier momento, sin esperar al martes (sección 10.3).

Si algún día se necesita cambiar la frecuencia o el horario, solo hay que editar esa línea. Algunos ejemplos:

```
- cron: '7 13 * * *' # todos los días, 8:07am Colombia
- cron: '7 13 * * 1-5' # lunes a viernes, 8:07am Colombia
- cron: '7 13 1 * *' # el día 1 de cada mes, 8:07am Colombia
- cron: '0 6 * * 1' # todos los lunes, 6:00am Colombia
```

Después de cambiar el cron, hay que hacer commit y push del archivo — GitHub vuelve a leer el horario automáticamente, no hay que "reactivar" nada aparte.

Ojo: GitHub Actions no garantiza que el cron dispare exactamente al segundo (ni siquiera al minuto) — en momentos de mucha carga puede haber unos minutos de retraso. Es normal.

10.3 Cómo correrlo manualmente

Si no se quiere esperar al próximo martes para probar un cambio:

1. Entra al repositorio en GitHub: https://github.com/dirplaneacionun/snies_monitor_posgrado
2. Pestaña **Actions** (arriba).
3. En la lista de la izquierda, clic en **SNIES Posgrado Report**.
4. Botón **Run workflow** (a la derecha) → **Run workflow** de nuevo para confirmar.
5. Se abre una corrida nueva; se puede hacer clic en ella para ver los logs en vivo, paso a paso.

Esto es exactamente lo mismo que pasaría un martes a las 8:07am — corre el pipeline completo, envía el correo real, y publica el dashboard real.

No es un modo de "prueba" sin efecto: si se corre manualmente, sí le llega correo de verdad a los destinatarios configurados y sí se actualiza la página pública.

10.4 Qué hace cada paso del workflow

```
jobs:
  run-snies-posgrado:
    runs-on: ubuntu-latest      # el "computador temporal" es Linux
    timeout-minutes: 60        # si se demora más de 1 hora, se cancela solo

    steps:
      - Checkout                # 1. Descarga el código del repositorio
      - Instalar Chrome          # 2. Instala Google Chrome (necesario para Selenium)
      - Setup Python             # 3. Instala Python 3.11
      - Instalar dependencias    # 4. pip install -r requirements.txt
      - Ejecutar pipeline posgrado # 5. python scripts/run_snies_posgrado.py
                                   # (descarga + compara + gráficos + envía correo, todo junto)
      - Subir screenshots de debug # 6. sube tmp/*.png como artifact (siempre, incluso si falló)
      - Commitear datos          # 7. git add + git commit de data/ y Programas/ (SIN push todavía)
      - Generar dashboard        # 8. python docs/generar_dashboard.py
      - Commitear dashboard y publicar # 9. git add docs/ + git commit + AQUÍ SÍ git push (de todo junto)
```

⚠ **Detalle importante que no es obvio a simple vista:** el paso 7 ("Commitear datos") hace `git commit`, pero **no** hace `git push`. El único `git push` de todo el workflow está al final, en el paso 9, y sube **de una sola vez** tanto el commit de datos (paso 7) como el commit del dashboard (paso 9). Esto quiere decir que si el paso 8 (generar el dashboard) llegara a fallar por cualquier razón, el commit de datos del paso 7 **se queda guardado únicamente dentro del computador temporal de esa corrida**, que se destruye al terminar el workflow — es decir, en ese escenario los datos nuevos detectados ese día **no llegarían a subirse al repositorio real**, aunque el log diga que sí se "commitearon". Vale la pena tenerlo en cuenta si algún día se ve que una corrida falló después del paso "Ejecutar pipeline posgrado": conviene revisar si el dashboard sí llegó a generarse y publicarse, no asumir que los datos ya quedaron a salvo solo porque el paso de "Commitear datos" se vio verde en el log.

Requisito importante para que el `git push` funcione en general: el repositorio debe tener activado, en `Settings → Actions → General → Workflow permissions`, la opción **"Read and write permissions"**. Si esa opción está en modo solo lectura, el pipeline completo corre bien (descarga, compara, envía el correo) pero el paso final de guardar los cambios fallará con un error de permisos. Si algún día se ve que el correo llega pero el dashboard nunca se actualiza (ni siquiera los datos), este es el primer lugar para revisar.

Nota técnica menor: el workflow define `FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true` como variable de entorno del job. Esto simplemente obliga a que las "acciones" de GitHub basadas en JavaScript (como el propio checkout o la subida de artifacts) usen una versión más nueva de Node.js por dentro — es un ajuste de infraestructura, no algo que haya que tocar nunca manualmente.

11. Los "Secrets" de GitHub

Los *Secrets* son la forma en que GitHub Actions maneja información sensible (contraseñas, tokens) sin que quede expuesta en el código fuente. Un secret se define una sola vez desde la configuración del repositorio, y el workflow lo referencia con `${{ secrets.NOMBRE }}` sin que su valor real aparezca nunca en los logs (GitHub lo enmascara automáticamente si por error se intentara imprimir).

Este proyecto usa 3 secrets propios, más uno automático que provee GitHub:

SECRET	PARA QUÉ SE USA	¿AUTOMÁTICO?
SMTP_USER	El correo que envía los reportes (ej. <code>monitor.snies@gmail.com</code>)	No, hay que crearlo
SMTP_PASS	La contraseña (de aplicación) de esa cuenta de correo	No, hay que crearlo
DESTINATARIOS	Lista de correos que reciben el reporte, separados por coma	No, hay que crearlo
GITHUB_TOKEN	Token temporal que usa el workflow para hacer <code>git push</code> de vuelta al repositorio, y para el <code>checkout</code>	Sí , GitHub lo genera solo en cada corrida — no hay que crearlo ni renovarlo nunca

Cómo ver o cambiar un secret

1. Entra a `https://github.com/dirplaneacionun/snies_monitor_posgrado` (se necesitan permisos de administrador o de escritura en el repositorio).
2. Pestaña **Settings**.
3. Menú de la izquierda: **Secrets and variables** → **Actions**.
4. Ahí se ve la lista: `SMTP_USER` , `SMTP_PASS` , `DESTINATARIOS` .

Importante: por seguridad, GitHub nunca deja ver el valor actual de un secret una vez guardado — solo se puede **sobrescribir** con un valor nuevo (botón "Update"). Si no se sabe qué correo o contraseña hay configurados actualmente, no hay forma de consultarlo desde la interfaz; hay que preguntarle a quien lo haya configurado, o simplemente definir uno nuevo.

No hace falta tocar ningún archivo de código ni "redespigar" nada al cambiar un secret — la próxima vez que el workflow corra (el martes siguiente, o manualmente) ya usará el valor nuevo automáticamente.

12. Cómo cambiar los destinatarios del correo

1. Ve a `Settings` → `Secrets and variables` → `Actions` en el repositorio (sección 11).
2. Busca el secret `DESTINATARIOS` y haz clic en el lápiz/**Update**.
3. Escribe la lista completa de correos separados por comas. No importa si dejas espacios después de la coma — el código les hace `strip()` (les quita los espacios sobrantes) automáticamente:

```
persona1@uninorte.edu.co, persona2@uninorte.edu.co, decano.negocios@uninorte.edu.co
```

4. Guarda ("Update secret").

Eso es todo — **la lista se reemplaza completa**, no se "agrega" un correo a lo que ya había. Si se quiere agregar una persona sin quitar a nadie, hay que escribir la lista completa de nuevo (las que ya estaban + la nueva).

Recuerda: este cambio no se ve reflejado hasta la próxima corrida del workflow (el próximo martes 8:07am, o antes si alguien lo corre manualmente).

13. Cómo cambiar la cuenta de correo que envía los reportes

Procedimiento completo para el día en que haya que migrar a otra cuenta (porque la actual se va a dar de baja, o cambia quién administra el monitor).

13.1 Si se sigue usando Gmail (caso más común)

Gmail **no permite** que un programa inicie sesión con la contraseña normal de la cuenta por seguridad — hay que generar una "**contraseña de aplicación**" (App Password), que es una clave especial de 16 caracteres solo para este uso.

1. Entra a la cuenta de Gmail que se va a usar para enviar los reportes.
2. Activa la **verificación en dos pasos** si no la tiene activada (obligatorio: sin esto, Google ni siquiera muestra la opción de contraseñas de aplicación). Se hace desde `myaccount.google.com` → Seguridad .
3. Ve a `myaccount.google.com/apppasswords` (o busca "Contraseñas de aplicaciones" dentro de Seguridad).
4. Crea una nueva, ponle un nombre que la identifique (ej: "SNIES Monitor Posgrado GitHub Actions").
5. Google muestra una clave de 16 caracteres (sin espacios) — **cópiala en ese momento**, porque después no se puede volver a ver.
6. En GitHub, ve a `Settings` → `Secrets and variables` → `Actions` (sección 11) y actualiza:
 - `SMTP_USER` → el nuevo correo completo (ej: `nuevo.monitor@gmail.com`)
 - `SMTP_PASS` → la contraseña de aplicación de 16 caracteres recién generada (**no** la contraseña normal de la cuenta)
7. Listo — no hay que tocar código, porque el host (`smtp.gmail.com`) y el puerto (`587`) siguen siendo los mismos.

13.2 Si se cambia a un proveedor distinto de Gmail (Outlook/Office 365, correo institucional propio, etc.)

Aquí sí hay que editar código, porque el servidor SMTP está escrito directamente en `scripts/send_report_posgrado.py` :

```
# scripts/send_report_posgrado.py
SMTP_HOST = "smtp.gmail.com"
SMTP_PORT = 587
```

Pasos:

1. Averiguar el host y puerto SMTP del nuevo proveedor. Algunos ejemplos comunes:

PROVEEDOR	HOST	PUERTO
Gmail	<code>smtp.gmail.com</code>	<code>587</code>
Outlook / Office 365	<code>smtp.office365.com</code>	<code>587</code>
Zoho Mail	<code>smtp.zoho.com</code>	<code>587</code>

2. Editar esas dos líneas en `scripts/send_report_posgrado.py` con el host/puerto correcto.
3. Hacer commit y push del cambio (esto sí requiere subir código, a diferencia de solo cambiar un secret).
4. Actualizar los secrets `SMTP_USER` y `SMTP_PASS` en GitHub con las credenciales de la cuenta nueva (sección 11). Ojo: algunos proveedores (como Office 365) pueden requerir además activar "SMTP AUTH" en la configuración de la cuenta antes de que funcione.

13.3 Cómo comprobar que quedó bien configurado

La forma más simple es correr el workflow manualmente (sección 10.3) y revisar el log del paso "Ejecutar pipeline posgrado" — si algo falla en el envío, aparecerá algo como:

```
ERROR ... Error enviando el correo.
smtplib.SMTPAuthenticationError: (535, b'5.7.8 Username and Password not accepted...')
```

Ese error específico casi siempre significa: se usó la contraseña normal de Gmail en vez de una contraseña de aplicación, o el usuario/contraseña están mal escritos en los secrets.

Si se prefiere probar sin esperar a que corra el scraping completo (que se demora varios minutos), se puede correr localmente con datos ya descargados — ver sección 15.

14. GitHub Pages: cómo se publica la página web

GitHub Pages es un servicio gratuito de GitHub que convierte archivos HTML de un repositorio en un sitio web público, sin necesidad de contratar hosting.

Cómo verificar/activar la configuración

1. **Settings** → **Pages** en el repositorio `snies_monitor_posgrado`.
2. En **"Source"** debe estar seleccionado: **Deploy from a branch**.
3. En **"Branch"**: `main` y la carpeta `/docs`.
4. Guardar si se hizo algún cambio.

Con eso, cualquier archivo dentro de `docs/` que se suba a la rama `main` queda publicado automáticamente (normalmente en 1-2 minutos) en una URL con este patrón:

```
https://dirplaneacionun.github.io/snies_monitor_posgrado/
```

Esta URL es la que correspondería según el nombre de la organización (`dirplaneacionun`) y del repositorio (`snies_monitor_posgrado`) — conviene confirmarla directamente en **Settings** → **Pages** , donde GitHub muestra la URL exacta una vez publicada por primera vez.

No hay que hacer nada más — el paso "Commitear dashboard y publicar" del workflow (sección 10.4) ya se encarga de subir la carpeta `docs/` actualizada cada vez que corre. A diferencia del correo (sección 8), el correo de este proyecto **no incluye actualmente un botón o enlace** hacia el dashboard — solo trae las tablas y los Excel adjuntos.

15. Cómo correr todo esto en tu propio computador

Sirve para probar cambios antes de subirlos, o para depurar un problema sin tener que esperar al workflow de GitHub.

15.1 Preparar el entorno

```
git clone https://github.com/dirplaneacionun/snies_monitor_posgrado.git
cd snies_monitor_posgrado

pip install -r requirements.txt
```

Las librerías usadas (`requirements.txt`) son:

LIBRERÍA	PARA QUÉ
<code>pandas</code>	Leer/escribir/comparar los Excel (la herramienta estándar de Python para manejar tablas de datos)
<code>openpyxl</code>	El "motor" que usa pandas por detrás para leer/escribir archivos <code>.xlsx</code>
<code>selenium</code>	Controlar el navegador Chrome automáticamente
<code>webdriver-manager</code>	Descarga automáticamente la versión correcta de ChromeDriver — no hay que instalarlo a mano
<code>matplotlib</code>	Dibuja los 2 gráficos PNG que se adjuntan al correo

15.2 Probar el pipeline SIN hacer scraping real (recomendado para probar cambios de lógica)

El proyecto trae un script hecho justo para esto: `scripts/test_pipeline.py`. En vez de abrir un navegador y descargar de internet, toma dos Excel que **ya** existen en `Programas/` (uno como "hoy" y otro como "anterior") y corre exactamente la misma lógica de comparación/clasificación/gráficos que correría en producción, pero guardando el resultado en una carpeta aparte (`tmp/test_pipeline_output/`) para no tocar ni ensuciar `data/novedades/`:

```
HOY_FILE      = ROOT / "Programas" / "Programas postgrado 27-08-25.xlsx"
ANTERIOR_FILE = ROOT / "Programas" / "Programas postgrado 20-08-25.xlsx"
TODAY = date(2025, 8, 27)

TEST_OUTPUT_DIR = ROOT / "tmp" / "test_pipeline_output" # nunca escribe en producción
```

Para correrlo:

```
python scripts/test_pipeline.py
```

Esto no envía ningún correo — solo imprime en pantalla cuántos nuevos/inactivos/modificados encontró y una vista previa de los primeros 5 de cada tipo. Es la forma más segura de comprobar que un cambio en `run_snies_posgrado.py` (por ejemplo, agregar un campo a `COLS_VIGILAR`) funciona como se espera, sin arriesgarse a mandar correos de prueba a la lista real ni a ensuciar el historial acumulado.

Si se quiere probar con fechas distintas, hay que editar esas tres líneas del script para apuntar a otro par de archivos que existan en `Programas/`.

15.3 Correr el pipeline completo (con scraping real)

```
python scripts/run_snies_posgrado.py
```

Si no se configuran las variables de entorno del correo, el script sigue funcionando igual — solo falla (sin romper nada) al intentar enviar el correo al final, y lo indica en el log.

15.4 Correr todo, incluyendo el correo real

```
# En Windows (PowerShell):
$env:SMTP_USER = "tu_correo@gmail.com"
$env:SMTP_PASS = "tu_contraseña_de_aplicacion"
$env:DESTINATARIOS = "tu_correo_de_prueba@gmail.com"
python scripts/run_snies_posgrado.py
```

⚠ Cuidado: esto envía un correo real a los destinatarios que se hayan puesto. Si solo se quiere probar el envío, hay que poner el propio correo en `DESTINATARIOS` en vez de la lista real de la universidad.

15.5 Ver el navegador en acción (en vez de invisible)

```
$env:SNIES_HEADLESS = "0"
python scripts/run_snies_posgrado.py
```

Por defecto Chrome corre invisible ("headless"). Con esta variable se puede ver la ventana abrirse y hacer clic en los filtros en vivo — muy útil para diagnosticar si el portal del SNIES cambió de diseño.

15.6 Regenerar el dashboard localmente

```
python docs/generar_dashboard.py
```

Esto lee lo que ya esté en `Programas/` y `data/novedades/` (tal como estén en la copia local del repositorio) y regenera todos los `.html` dentro de `docs/`. Después se puede abrir `docs/index.html` directamente con el navegador (doble clic) para revisar los cambios de diseño antes de subirlos.

16. Problemas comunes y cómo resolverlos

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
El workflow falla en el paso "Ejecutar pipeline posgrado" con un <code>TimeoutError</code> sobre la descarga	El portal del SNIES cambió de diseño (cambió el texto de algún filtro o del botón de descarga), o está caído/muy lento	Revisar las capturas <code>debug_snies.png</code> / <code>debug_post_filtros.png</code> (se descargan como artifact "debug-screenshots" desde la corrida fallida, en la pestaña Actions) para ver qué mostraba la página; puede que haya que actualizar los textos en <code>XPATHS</code> o en las llamadas a <code>_pf_select_radio</code> en <code>run_snies_posgrado.py</code>
El correo nunca llega, pero el dashboard sí se actualiza	Falló el login SMTP (contraseña vencida, se usó la contraseña normal en vez de la de aplicación, la cuenta bloqueó el acceso)	Revisar el log del paso "Ejecutar pipeline posgrado", buscar <code>Error enviando el correo</code> ; regenerar la contraseña de aplicación (sección 13.1)
El dashboard no se actualiza, y tampoco los datos, aunque el pipeline parece haber corrido bien	Falta el permiso "Read and write permissions" en <code>Settings → Actions → General → Workflow permissions</code> , o el paso "Generar dashboard" falló después de que los datos ya se habían "commiteado" localmente (ver el detalle de la sección 10.4 sobre el push único al final)	Activar el permiso de escritura; revisar el log completo de la corrida, no solo si el paso 5 se vio verde
El pipeline aborta diciendo "demasiados programas — probable descarga sin filtros"	El robot descargó el Excel sin que los filtros hayan surtido efecto (por lentitud del portal, por ejemplo), o el snapshot anterior en <code>Programas/</code> está corrupto	Revisar <code>debug_post_filtros.png</code> ; puede ser puntual y resolverse solo reintentando (correr el workflow manualmente de nuevo)
Aparecen muchísimos "Modificados" de golpe, que no parecen reales	El snapshot anterior contra el que se comparó estaba corrupto, incompleto, o venía de una descarga sin filtrar de hace tiempo	Revisar cuál fue el "snapshot anterior" usado (sale en el log: <code>Snapshot ANTERIOR: Programas postgrado DD-MM-YY.xlsx</code>) y comparar manualmente ese archivo
Un programa que se sabe que cambió no aparece como "Modificado"	El cambio fue en un campo que no está en <code>COLS_VIGILAR</code> (sección 6.5) — por ejemplo, cambió de nombre, de departamento, o de duración en periodos	Es el comportamiento esperado del sistema, no un error — si se quiere vigilar ese campo, hay que agregarlo a <code>COLS_VIGILAR</code>
Un programa aparece con <code>DIVISIÓN UNINORTE = "Sin clasificar"</code>	Su área CINE (<code>CINE_F_2013_AC_CAMPO_DETALLADO</code>) todavía no está mapeada en <code>Categorización divisiones SNIES.xlsx</code>	Abrir ese Excel (hoja <code>Hoja3</code>) y agregar la fila de traducción correspondiente
No hay permisos para ver/editar los Secrets en GitHub	Los secrets solo los puede administrar quien tenga rol de administrador (o "write", según configuración) en el repositorio	Pedirle a quien administre el repositorio en GitHub que dé acceso, o que haga el cambio

17. Cosas importantes que hay que saber (detalles no obvios)

Al revisar el proyecto a fondo se encontraron varios puntos que vale la pena tener presentes, porque no son evidentes con solo mirar el correo o el dashboard:

1. El correo se llama **"Reporte Mensual"**, pero en la práctica el workflow corre **semanalmente** (cada martes desde el ajuste más reciente). El texto **"Reporte Mensual SNIES – Posgrado"** viene de una etapa anterior del proyecto en la que se planeaba una frecuencia mensual, pero el cron configurado siempre ha correspondido a una frecuencia semanal (antes era todos los lunes; ahora es todos los martes a las 8:07am). Si

algún día se decide de verdad hacerlo mensual (por ejemplo, solo el primer lunes de cada mes), el cron tendría que ajustarse con una lógica adicional (los cron estándar no tienen una forma nativa de decir "el primer lunes del mes"; normalmente se logra poniendo el día `1-7` + día de la semana deseado y agregando una validación extra en el propio script). Mientras tanto, conviene saber que el asunto del correo no describe la frecuencia real.

2. El archivo de configuración se llama `snies_posgrado_daily.yml` ("daily" = diario), pero tampoco corre a diario — el nombre del archivo quedó de una versión anterior y no se actualizó cuando cambió la frecuencia real.
3. El `git push` de cada corrida es uno solo, al final de todo (ver el detalle completo en la sección 10.4). Si el paso de generar el dashboard fallara, los datos nuevos detectados ese día podrían no llegar a subirse al repositorio real, aunque el paso anterior de "Comitear datos" se haya visto exitoso en el log.
4. El archivo de categorización (`Categorización divisiones SNIES.xlsx`) vive en la raíz del repositorio, no dentro de `data/` , y su nombre real tiene un espacio justo antes de `.xlsx` (`"...SNIES .xlsx"` , no `"...SNIES.xlsx"`). Si algún día se reemplaza ese archivo o se sube una versión nueva, hay que conservar el nombre exacto (con el espacio) o, si se prefiere corregirlo, actualizar también la ruta `CAT_FILE` en `scripts/run_snies_posgrado.py` .
5. Los nombres de los snapshots usan la ortografía "postgrado" (con "t"), no "posgrado": `Programas postgrado DD-MM-YY.xlsx` . Es una inconsistencia menor de redacción dentro del propio proyecto, pero el sistema depende literalmente de ese patrón exacto para reconocer y ordenar los snapshots por fecha — no se debe "corregir" el nombre de un archivo existente sin actualizar también la expresión regular `_PROG_RE` en el código.
6. Este repositorio no tiene un `README.md` . Este manual (`MANUAL_POSGRADO.md` / `MANUAL_POSGRADO.pdf`) es, por ahora, la única documentación completa del proyecto.
7. Existe una carpeta `pregrado/` en este mismo repositorio, con archivos HTML viejos del dashboard de *pregrado* (de otro proyecto). Está excluida del control de versiones (`.gitignore`) y el propio `.gitignore` aclara que es "material de consulta, no del repo" — no la usa ningún script de este proyecto y se puede ignorar por completo.
8. No existe un archivo `dashboard_data.json` separado. A diferencia de otros diseños posibles, todos los datos del dashboard quedan incrustados directamente dentro de cada archivo `.html` que genera `docs/generar_dashboard.py` .
9. Las tres tablas del correo (Nuevos, Inactivos, Modificados) siempre muestran máximo 10 filas en el cuerpo del mensaje, sin excepción — para ver la lista completa de cualquiera de las tres siempre hay que abrir el Excel adjunto correspondiente.
10. La columna `Estado` (`Activo` / `Inactivo`) se calcula para cada fila de nuevos/inactivos/modificados, pero no se muestra en ningún lado (ni correo, ni dashboard) — solo queda disponible si se abre el Excel directamente.
11. El umbral de seguridad contra descargas corruptas es de 12.000 programas (no 10.000 como en el monitor de pregrado), porque el volumen normal de posgrado activo es distinto al de pregrado.

18. Glosario de términos técnicos

- **SNIES**: Sistema Nacional de Información de la Educación Superior — la base de datos pública del Ministerio de Educación de Colombia con todos los programas académicos del país.
- **Snapshot**: una "foto" del estado de los datos en un momento específico. Cada Excel en `Programas/` es un snapshot.
- **Scraping / Web scraping**: técnica de extraer información de una página web automatizando un navegador, cuando no existe una forma más directa (API) de obtener esos datos.
- **Selenium**: la librería de Python que controla un navegador real (Chrome, en este caso) de forma automática.
- **Headless**: modo en el que un navegador corre sin mostrar ninguna ventana en pantalla — funciona igual por dentro, pero no se ve.
- **XPath**: una forma estándar de "darle direcciones" a un elemento dentro de una página web (ej: "el botón que tiene un texto que dice tal cosa"), para que un programa lo pueda encontrar y hacer clic en él.
- **AJAX**: técnica con la que una página web le pide datos nuevos al servidor y actualiza solo una parte de la pantalla, sin recargar la página completa.
- **JSF / PrimeFaces**: una tecnología de programación web (común en sistemas del gobierno colombiano) que genera identificadores técnicos que cambian con cada actualización del sistema — por eso el robot evita depender de ellos y usa el texto visible.
- **SMTP**: el protocolo estándar de internet para enviar correos electrónicos por programación.

- **App Password / Contraseña de aplicación:** una clave especial que generan Gmail (y otros proveedores) para que un programa pueda enviar correos sin usar la contraseña normal de la cuenta, por seguridad.
- **GitHub Actions:** el servicio de GitHub para correr tareas automáticas (workflows) en la nube, programadas o manuales.
- **Workflow:** un archivo de configuración (`.yaml`) que le dice a GitHub Actions exactamente qué pasos ejecutar.
- **Cron:** la notación estándar para expresar "a qué hora y qué días" debe correr algo de forma automática.
- **Secret:** un valor sensible (contraseña, token) guardado de forma cifrada en GitHub, que un workflow puede usar sin exponerlo en el código.
- **GitHub Pages:** el servicio gratuito de GitHub para publicar páginas web estáticas directamente desde un repositorio.
- **CINE (Clasificación Internacional Normalizada de la Educación):** sistema de clasificación de áreas del conocimiento definido por la UNESCO, usado internacionalmente para comparar programas académicos de distintos países/sistemas.
- **Núcleo Básico del Conocimiento (NBC):** otra clasificación de áreas de conocimiento, propia del sistema educativo colombiano (distinta del CINE, aunque con propósito similar).
- **Dashboard:** panel visual con gráficos, indicadores y tablas para explorar datos de forma interactiva.
- **Plotly:** la librería de JavaScript que dibuja los gráficos interactivos del dashboard (zoom, valores exactos al pasar el mouse, etc.).
- **JSON:** formato de texto muy usado para representar datos estructurados (parecido a un diccionario de Python) — es el formato en el que cada página del dashboard lleva sus datos incrustados.
- **Matplotlib:** librería de Python usada para dibujar los 2 gráficos estáticos (PNG) que se adjuntan al correo.
- **Runner:** el computador temporal en la nube donde GitHub Actions ejecuta cada corrida del workflow; se crea y se destruye en cada ejecución.
- **Commit / Push:** dos pasos distintos de Git. "Commit" guarda un cambio localmente (en la copia del repositorio del runner); "push" es lo que efectivamente sube ese cambio al repositorio real en GitHub. Un commit sin push no le llega a nadie más — se explica por qué esto importa en la sección 17, punto 3.

Manual generado a partir de una revisión completa del código fuente del repositorio `snies_monitor_posgrado` (julio de 2026), incluyendo `scripts/run_snies_posgrado.py` , `scripts/send_report_posgrado.py` , `scripts/test_pipeline.py` , `analisis_historico_posgrado.py` , `docs/generar_dashboard.py` y `.github/workflows/snies_posgrado_daily.yml` . Si el código cambia de forma importante en el futuro, conviene revisar que este manual siga reflejando la realidad, especialmente las secciones 4, 6, 10 y 17.